Project ECHO: A Bayesian POMDP and Sigma-Lognormal Framework for Trust Calibration and Cognitive Load Adaptation in Agentic Human-Computer Interaction

Akshith Srinivas[1*], Abishek KG[1] and Anita Christaline J[1]

[1]School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, Tamil Nadu, India.

*Corresponding author(s). E-mail(s): akshith.srininvas2023@vitstudent.ac.in;
Contributing authors: abishek.kg2023@vitstudent.ac.in; anitachristaline.j@vit.ac.in;

Abstract

People are now entrusting important decisions to agentic AI systems. The success of the partnership between humans and their AI agents depends on two crucial factors that are generally invisible to the software: the degree to which the human relies on the agent and the mental effort that the human is required to exert on the task in question. Both of these factors can be measured by questionnaires and physiological sensors, but neither of these measurement devices fits into the typical desktop work environment. Project ECHO (Empathic Cognitive Hook Orchestrator) addresses this challenge by providing a means of estimating reliance, workload, and emotion by simply monitoring a person's use of their keyboard and mouse. Project ECHO utilizes three distinct models to estimate these factors. A Bayesian partially observable Markov decision process (POMDP) is used to estimate the probability distribution of 15 hidden states of human behavior, where each state represents a combination of one of five levels of reliance on the agent and one of three levels of cognitive workload. The movements of the mouse pointer can be decomposed into the lognormal speed components described by the kinematic theory of rapid human movement; the fit of these speed components to the actual movements of the pointer is used both as a measure of strain on the user and as a filter for the movements that are not to be used as measures of reliance or workload. Finally, a dual-stream Transformer model (DSTER) is used to analyze the pattern of a person's keystrokes to estimate

their emotional state; the DSTER model was tested on 199 labelled windows of human-computer interaction, achieving an accuracy rate of 95.98%, a macro F1 score of 0.9496, and a Cohen kappa of 0.9481. The estimates from these three models were used to adjust the difficulty of 20 settings in four different game scenarios; the estimates and the control loop that responded to them occurred in less than 40 ms. Thus, Project ECHO demonstrates that the input devices that people use to interact with computers already contain enough information to allow the computer's user interface to adjust in response to the user's trust in the AI agent and the workload required of the user.

Keywords: Human-computer interaction, Bayesian POMDP, Sigma-Lognormal model, Trust calibration, Cognitive load, Affective computing, Dynamic difficulty adjustment, Keystroke dynamics, Transformer

1 Introduction

Anyone who has ever worked alongside a decision aid knows it can fail them in two dissimilar ways. When people rely on it too much, its errors go unnoticed; the literature defines this as automation bias. When people are distrustful, they ignore advice that would have helped, which is usually characterised as algorithm aversion [1, 2]. Either way, the human and the machine together are likely to do a worse job than they could. A major part of the issue can be traced to a design habit: the trust placed in the system is treated as fixed at deployment, when in practice it is a belief held by the user that rises and falls over the course of the engagement between the two entities [3, 4].

Project ECHO tackles these issues by focusing on the input of the user. Project ECHO uses the input from the keyboard and mouse of the user. Project ECHO is able to estimate the trust, cognitive demand, and emotional state of the user based on how the user is typing on the keyboard and using the mouse. Project ECHO has three models that allow for these estimations. The first model is a Bayesian POMDP [5]. This model maintains the joint belief of the 15 trust and cognitive demand combinations of the user. The second model uses the Sigma-Lognormal formulation of the kinematic theory [6, 7] to estimate the motor fatigue and tension of the user. The third model is DSTER. This model uses a Transformer architecture [8] with two attention streams to estimate the emotional state of the user based on the timing of their keystrokes [9].

These estimations from the three models are passed on to a middleware layer that is grounded in human-centred explainable AI (HCXAI) [10, 11]. Depending upon the estimations of the user, Project ECHO will perform certain actions. For instance, Project ECHO could change the display when the user is experiencing high levels of cognitive demand, provide more detailed explanations when the user is exhibiting high levels of trust in the system, or change the difficulty of the tasks that the user must perform. Project ECHO performs these actions in a way that is visible to the user and relatively subtle. Project ECHO aims to help the user with their tasks, but does not want to covertly nudge them to perform certain tasks.

This paper makes five main contributions to the field of AI and human-computer interaction. First, Project ECHO frames the issue of trust calibration as a belief-tracking problem in a POMDP. Second, Project ECHO uses the Sigma-Lognormal model in live mouse tracking and develops a tiered acceptance rule. Third, the two-stream Transformer model used by Project ECHO achieves an accuracy of 95.98% in predicting the emotional state of the user based on their keystrokes. Fourth, Project ECHO is tested in four game environments of different genres. Finally, the code, configuration files, and telemetry for Project ECHO are released.

The remainder of the paper will present the following information. Section 2 will describe the aims of Project ECHO. Section 3 will discuss the research that has been conducted previously in this area. Section 4 will detail the models that have been implemented in Project ECHO. Section 5 will describe the software that was implemented in Project ECHO. Section 6 will present the results of the experiments that were performed using Project ECHO. Section 7 will discuss the results, Section 8 will detail the limitations of Project ECHO, and Section 9 will conclude the paper.

2 Design Goals

Project ECHO grew out of the gaps we found in existing adaptive and affect-aware systems, with each part addressing a different one. Table 1 shows how each goal maps to the mechanism designed to address it.

Table 1: Goals that shaped Project ECHO and the mechanism used to meet each one.

Goal	Mechanism in ECHO
Track changes in user trust	Joint trust and demand state belief filter (15 states)
Measure workload without additional sensors	Typing latency with the CLC statistic
Counter over-reliance and underuse	Explanation detail set by inferred trust and stakes
Read emotion from the way people type	DSTER, a two-stream Transformer on keystroke characteristics
Read motor condition from the pointer	Lognormal decomposition of mouse speed profiles
Test the loop in a time crunch	Real-time control of 20 settings in 4 game scenarios

The guiding principle was simple: ECHO should help users reach an appropriate level of trust without steering them toward more or less trust. When users seem confused, doubtful, or overly reliant, ECHO adjusts both the amount and the form of its explanation. Dark patterns, invented urgency and other hidden forms of persuasion are excluded by design. ECHO should be useful and transparent about its actions, without steering people toward any particular choice.

3 Background and Related Research

3.1 Trust Between People and Automated Agents

Trust in automation means expecting an agent to help reach goals when things stay uncertain and risk is involved. Lee and See [3] put it that way. Hoff and Bashir [2] sorted experiments into dispositional, situational and learned trust. Reliability, transparency and past experience all shape it according to them.

Getting trust wrong leads to costs that show up in outcomes. Dzindolet [1] found relying too much hurts results as much as rejecting the system outright. The automation types from Parasuraman and others [4] keep trust mostly in the background. It does not shift moment to moment there.

Chen [12] proposed SAT (Situation Awareness-based Agent Transparency) so the agent shows reasoning, projections and uncertainty. People can then use it at the right level. What it misses is any check on whether trust is actually present right now. ECHO works from the other side by guessing a posterior over user trust from actions alone. It leaves transparency untouched. Maybe that fills the gap but I am not totally sure how it plays out in practice.

3.2 Measuring Cognitive Load

Sweller [13] proposed dividing cognitive load into intrinsic, extraneous, and germane categories. Wickens [14] later proposed a theory of multiple resources to explain multitasking behavior. The main difficulty with these projects has been obtaining information, since measurement has relied on self-reported and physical measurements [15]. Vizer, Zhou, and Sears [16] showed that cognitive and physical stressors can be measured using keystroke dynamics and in the language people type. That is the main reason to gather typing latency data.

3.3 Recognising Emotion from Behaviour

Picard [17] pioneered affective computing. Calvo and D'Mello [18] later reviewed methods and models used in affect detection. Epp, Lippold, and Mandryk [9] found that keystroke information recorded during typical computer use can be used to identify emotions. Models of attention for sequences [8, 19] have since contributed to advances in methods for analyzing sequential data. The DSTER method can be considered a contribution to these advances: one of its streams investigates how each keystroke feature evolves over time, and the other investigates the relationships between features within a single moment.

3.4 Lognormal Models of Rapid Movement

According to the kinematic theory of rapid human movements, the speed profile of a well-practised movement can be written as the vector sum of lognormal impulse responses produced by the neuromuscular system [6, 7]. The Sigma-Lognormal model built on this idea has been used widely to analyse and synthesise handwriting and signatures [20], and departures from clean lognormal behaviour have been tied to

fatigue, age and neuromotor disorders. Nearly all of that work processes recordings after the fact. ECHO instead fits the model to mouse movements while the user is still working.

3.5 Partially Observable Decision Models in HCI

POMDPs give a formal way to choose actions in sequence when the true state of the environment is hidden from the decision maker [5]. Robotics and spoken dialogue systems use them routinely, yet they appear rarely in work on interfaces that manage user trust. In ECHO the POMDP refines its belief about the user from passively collected evidence, so the user is never stopped and asked to rate their own state.

3.6 Adaptive Difficulty in Games

Hunicke [21] argued for dynamic difficulty adjustment (DDA) as a way to hold players in the flow channel described by Csikszentmihalyi [22]. Well-known commercial instances are the Director of Left 4 Dead, the hidden menace meter of Alien: Isolation and the concealed rank score of Resident Evil 4, and Yannakakis and Togelius [23] give a broader account of adaptation driven by player experience. These systems usually react to the performance of the player but not to the emotions felt by the player. ECHO relates the game parameters to the detected emotions and workload, and the time pressure of the game makes it an extremely demanding test of the inference process.

4 Inference Models

4.1 The Adaptation Loop

Fig. 1 shows ECHO operating in a loop with four phases that repeat. There is also a background listener built with `pynput`, a Python module for logging timestamped keyboard and mouse activity. These raw data are transformed into features: features related to the inter-keystroke interval (IKI), the Cognitive Load Correlation (CLC) and fitted Sigma-Lognormal parameters. In the third stage DSTER assigns an emotion label and the POMDP revises its joint belief about trust and demand. Last, the middleware converts these estimates into interface changes or difficulty changes. The user reacts to those changes, the reaction shows up in the next round of input, and the loop starts again.

4.2 Formulation as a POMDP

We describe the exchange between user and agent with the usual tuple $\langle \mathcal{S}, \mathcal{A}, T, \mathcal{O}, \Omega, R, \gamma \rangle$. Here $\mathcal{S}$ denotes the hidden states, $\mathcal{A}$ the actions, T the state dynamics, $\mathcal{O}$ the possible observations, Ω the likelihood of an observation given a state, R the reward and γ the discount. This paper is concerned with the inference side of the tuple, namely $\mathcal{S}$, T, $\mathcal{O}$ and Ω, since these alone fix the belief that the middleware consumes.

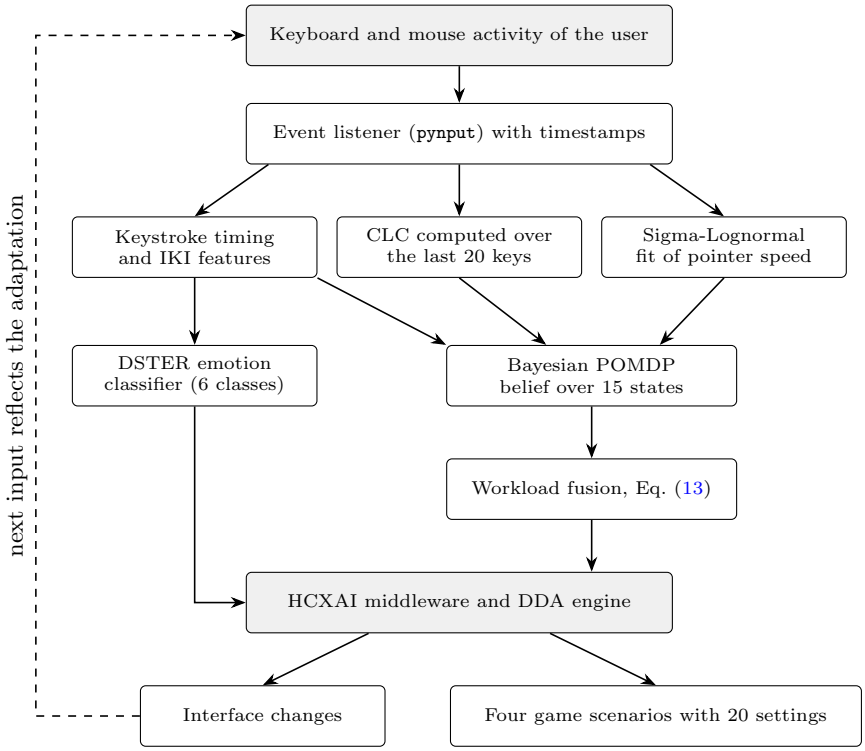


Fig. 1: Overview of the ECHO loop. Raw events become timing, CLC and pointer features. These feed DSTER and the belief filter, whose outputs are merged into one workload score and handed to the middleware and the difficulty engine. Whatever the user does next re-enters the loop as fresh input.

Hidden states.

Each hidden state combines one trust level with one demand level, so the state space is the Cartesian product

$$\mathcal{S} = \mathcal{T} \times \mathcal{C}, \qquad \mathcal{T} = \{\mathrm{VL}, \mathrm{L}, \mathrm{M}, \mathrm{H}, \mathrm{VH}\}, \qquad \mathcal{C} = \{\mathrm{L}, \mathrm{M}, \mathrm{H}\}, \tag{1}$$

where $\mathcal{T}$ holds the trust levels and $\mathcal{C}$ the demand levels, giving $|\mathcal{S}| = 15$. A state is written $s = (t, c)$, with $t \in \{0, 1, \ldots, 4\}$ indexing trust and $c \in \{0, 1, 2\}$ indexing demand.

Observations.

In every cycle the system receives the observation vector

$$\mathbf{o} = \left(\mathrm{CLC}_{\mathrm{raw}},\ \overline{\mathrm{IKI}},\ \mathrm{SNR}\right)^{\top}, \tag{2}$$

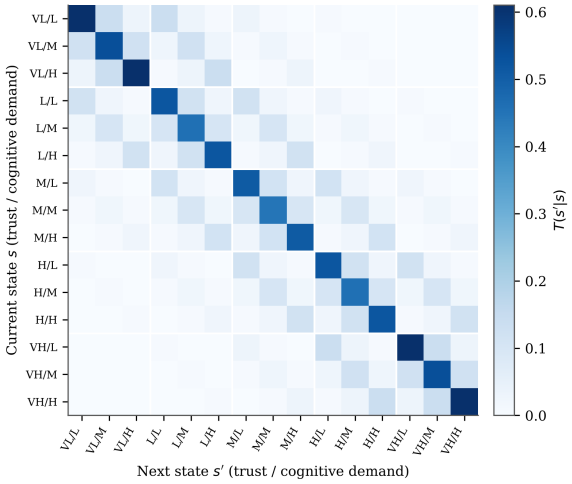


Fig. 2: The 15×15 kernel of Eq. (3) for $\alpha = 1.5$. Axis labels give the trust level (VL to VH) followed by the demand level (L, M, H). Most of the mass is on the diagonal and on the surrounding cells, similar to the gradual drift expected of trust and workload.

that is, the Cognitive Load Correlation, the average gap between keystrokes in milliseconds and the signal-to-noise ratio, in decibels, of the latest Sigma-Lognormal reconstruction.

4.2.1 State Dynamics

Neither trust nor workload tends to jump from one extreme to the other between two cycles. We therefore let the probability of moving to another cell of the state grid fall off exponentially with the number of steps taken along each axis:

$$T(s' \mid s) = \frac{\exp[-\alpha\,(d_t(s, s') + d_c(s, s'))]}{\sum_{s'' \in \mathcal{S}} \exp[-\alpha\,(d_t(s, s'') + d_c(s, s''))]}, \tag{3}$$

in which d_t and d_c are the index distances on the trust and demand scales, respectively, and $\alpha = 1.5$. The full 15×15 matrix can be found in Fig. 2. For the center cell (medium trust, medium demand), the probability of staying in the same cell is 0.447, the probability of moving to an adjacent cell is 0.100, and moves of two or more spaces on both axes have probabilities less than 0.01. The kernel strongly supports maintaining the current belief but also permits the belief to change when the user's state really does change.

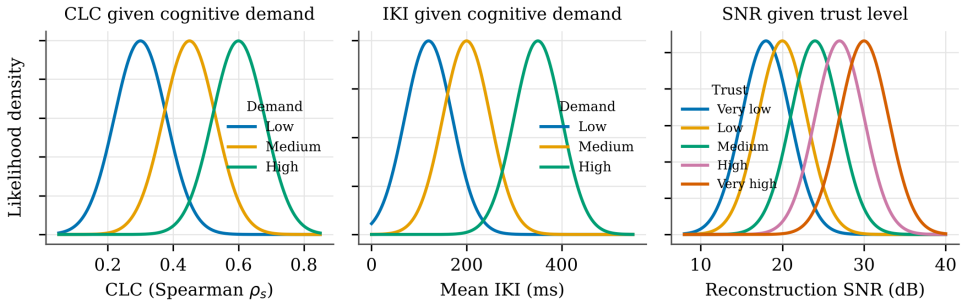


Fig. 3: Likelihood curves, as per Table 2. The left two panels represent CLC and mean IKI at each level of demand; the right panel represents reconstruction SNR at each level of trust.

4.2.2 Likelihood of an Observation

We assume that observations in the three channels are conditionally independent given information about the state. The likelihood is the product of three Gaussians:

$$P(\mathbf{o} \mid s) = \prod_{d \in \{\text{clc, iki, snr}\}} \frac{1}{\sqrt{2\pi}\,\sigma_d^{(s)}} \exp\left[-\frac{\left(o_d - \mu_d^{(s)}\right)^2}{2\left(\sigma_d^{(s)}\right)^2}\right], \tag{4}$$

CLC and IKI are indicators of the demand portion of the state. SNR is an indicator of the trust portion of the state. Table 2 gives the parameter values. Fig. 3 gives the related densities. Demand levels are best compared using measurements of IKI. The SNR trust curves have a high degree of similarity and offer little information about trust in a single cycle, and it may take many cycles for the filter to gather enough evidence to make a decision.

Table 2: Parameters of the Gaussian likelihoods in Eq. (4). Means are listed from the lowest to the highest level of the state component on which each channel depends.

Channel	Depends on	Means $\mu_d^{(s)}$	Spread σ_d
CLC	Demand (L, M, H)	0.30, 0.45, 0.60	0.08
IKI (ms)	Demand (L, M, H)	120, 200, 350	50
SNR (dB)	Trust (VL, L, M, H, VH)	18, 20, 24, 27, 30	3

4.2.3 Updating the Belief

When a new observation $\mathbf{o}$ is made, belief state b is updated using the dynamics and weighted by the likelihood:

$$b'(s') = \eta\, P(\mathbf{o} \mid s') \sum_{s \in \mathcal{S}} T(s' \mid s)\, b(s), \tag{5}$$

where η is a normalizing constant that makes b' sum to one,

$$\eta = \left[\sum_{s' \in \mathcal{S}} P(\mathbf{o} \mid s') \sum_{s \in \mathcal{S}} T(s' \mid s)\, b(s) \right]^{-1}. \tag{6}$$

Each time we perform such a step, we add a Laplace value of $\varepsilon = 10^{-5}$ to the result and normalise the result. Otherwise, one inaccurate measurement could bring the corresponding state to zero, and our filter would not be able to restore it. For the middleware part we first convert the belief into its two marginal distributions,

$$p(t) = \sum_{c=0}^{2} b(s_{t,c}), \qquad p(c) = \sum_{t=0}^{4} b(s_{t,c}), \tag{7}$$

and then compute the average values of trust and demand,

$$\bar{\tau} = \frac{1}{4} \sum_{t=0}^{4} t\, p(t), \qquad \bar{\kappa} = \frac{1}{2} \sum_{c=0}^{2} c\, p(c). \tag{8}$$

Here, division by 4 and by 2 is used to scale these values to the range $[0, 1]$.

4.3 Pointer Kinematics with the Sigma-Lognormal Model

The speed of the pointer on each move is represented as a sum of N lognormal factors:

$$v(t) = \sum_{i=1}^{N} \frac{D_i}{\sqrt{2\pi}\,\sigma_i\,(t - t_{0i})} \exp\left[-\frac{\left[\ln(t - t_{0i}) - \mu_i\right]^2}{2\sigma_i^2} \right], \qquad t > t_{0i}, \tag{9}$$

Each factor i has D_i as its magnitude, t_{0i} as the time at which it starts, μ_i as its log delay and σ_i as its log response time. These values are optimised by the use of L-BFGS-B, which is a quasi-Newton technique that is constrained by a box. In order to ensure that the optimization process takes place within the latency constraint, each trajectory is initially resampled to 200 points. How well the fit matches the recorded speed is summarised as

$$\mathrm{SNR}_{\mathrm{dB}} = 10 \log_{10} \left[\frac{\sum_j \left(v_j^{\mathrm{obs}}\right)^2}{\sum_j \left(v_j^{\mathrm{obs}} - v_j^{\mathrm{rec}}\right)^2} \right], \tag{10}$$

where v_j^{obs} is the speed obtained for sample j and v_j^{rec} is the speed reconstructed using the components fit to the data at sample j.

Data on mouse motion acquired through actual tasks are much noisier than data on pen trajectories collected on a tablet. That is why we assign to each trajectory a weight, which depends on the accuracy of its reconstruction (Table 3). Trajectories whose reconstruction is less than 15 dB are discarded.

Table 3: How a trajectory is weighted according to its reconstruction SNR.

SNR range (dB)	Fit quality	Weight	Treatment
< 15	Unsatisfactory	0.0	Excluded
15 to 20	Acceptable	0.5	Partial weight
20 to 25	Good	0.7	Normal weight
> 25	Excellent	1.0	Full weight

4.4 The Cognitive Load Correlation

CLC raises the fundamental question of whether a person's slowdown is due to the difficulty of the words they type. The question is answered by Spearman's rank correlation between the lexical complexity level of a word and the time interval between its keystrokes, calculated within a sliding window of the last 20 keystrokes. Letting $\mathbf{x}$ be the complexity levels and $\mathbf{y}$ be the corresponding keystroke time intervals within the window, we obtain

$$\text{CLC} = \rho_s\left(\text{rank}(\mathbf{x}),\ \text{rank}(\mathbf{y})\right), \tag{11}$$

In practice the value does not go far outside the range 0.35 to 0.55. When it rises above that band, typing pace is being set more and more by how difficult the material is, and we read this as a sign of increased demand.

4.5 DSTER: Two Attention Streams over Keystrokes

DSTER takes a sequence of keystroke feature vectors and passes it through two Transformer encoders running side by side, each attending along a different axis (Table 4). Every keystroke contributes four numbers: how long the key is held (dwell), the interval since the previous key, key velocity, and the flight time between releasing one key and pressing the next. One encoder treats time as its sequence dimension and so learns the rhythm of each feature. The other treats the features as its sequence dimension and so learns how they co-vary at a given instant. We concatenate the two embeddings, an operation written $\|$ below, and pass them through a small multilayer perceptron that outputs a 64-dimensional code,

$$\mathbf{e} = \text{MLP}_{\text{fuse}}\left([\mathbf{e}_{\text{time}} \,\|\, \mathbf{e}_{\text{feat}}]\right) \in \mathbb{R}^{64}, \tag{12}$$

and a softmax layer maps this code to one of six labels: angry, focused, thinking, neutral, joy and given up.

Table 4: Roles of the two DSTER encoders.

Encoder	Sequence axis	What it learns
Time encoder	Keystrokes in order	How the timing of each feature fluctuates across the window
Feature encoder	The four features	How dwell, interval, velocity and flight relate within one keystroke

4.6 Combining Workload Signals

No single workload cue is trustworthy on its own, so the score that the middleware sees blends three of them: the demand marginal from the POMDP, the raw CLC, and a motor load factor derived from the lognormal parameters,

$$\mathrm{CL}_{\mathrm{fused}} = 0.5\,\mathrm{CL}_{\mathrm{POMDP}} + 0.3\,\mathrm{CLC}_{\mathrm{raw}} + 0.2\,\mathrm{NM}_{\mathrm{load}}, \tag{13}$$

Half the score is based on the POMDP estimate, which has already averaged information across channels and over time.

4.7 How the System Responds

The middleware follows fixed rules to promote consistency and to restrict its behaviour ethically. Regarding trust, when a user appears to place high trust in the agent while making an important decision, more information is given to the user so that they evaluate the recommendation with skepticism; when trust appears low, the explanation is kept short and to the point. In terms of workload, when $\mathrm{CL}_{\mathrm{fused}}$ exceeds 0.7, the interface removes pictures and other nonessential material and shortens some text. Emotion-driven changes to tone and pacing are made only when DSTER confidence is greater than 60%.

All decisions made by the middleware are captured in JSON format showing the interface configuration changes and verbosity settings, so that interface activity can be monitored and audited afterwards. Downstream components do not have access to raw biometric data, and the system is not allowed to make task decisions for the user. If the spread of the fitted σ parameters shifts by more than 0.15, we classify pointer behaviour as highly irregular, record it as a neuromotor stress event and simplify the screen. And when predicted states start to disagree more often with the states observed afterwards, the middleware adapts less frequently, on the reasoning that a model which is unsure of itself should act less.

For games, the DDA engine does not pick the single likeliest state. Writing $\Delta p^{(s)}$ for the change a rule would make to parameter p if the state were known to be s, the change actually applied is the expectation under the current belief,

$$\Delta p = \sum_{s \in \mathcal{S}} b(s)\,\Delta p^{(s)}, \tag{14}$$

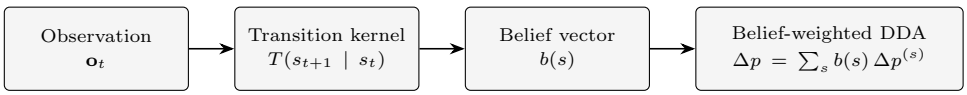


Fig. 4: Transition path from observation to difficulty adjustment. The new observation pushes the belief through the transition kernel, and the belief-weighted average of state-adjustment terms in Eq. (14) determines the shift for each parameter.

Algorithm 1 A single pass of the ECHO loop

Require: current belief b, kernel T, likelihood parameters $\{\mu_d^{(s)}, \sigma_d^{(s)}\}$
Collect key and pointer events for the current window
Compute $\overline{\text{IKI}}$ and CLC by Eq. (11), and fit Eq. (9) to every trajectory
Obtain SNR from Eq. (10) and assign weights from Table 3
$\mathbf{o} \leftarrow (\text{CLC}_{\text{raw}}, \overline{\text{IKI}}, \text{SNR})$
for each $s' \in \mathcal{S}$ **do**
    $b'(s') \leftarrow P(\mathbf{o} \mid s') \sum_{s \in \mathcal{S}} T(s' \mid s)\, b(s)$
end for
Normalise b' with η from Eq. (6) and add Laplace mass $\varepsilon = 10^{-5}$
Obtain $\bar{\tau}$ and $\bar{\kappa}$ by Eqs. (7) and (8), and CL_{fused} by Eq. (13)
$(\hat{y}, c_e) \leftarrow$ DSTER(keystroke window) ▷ label and confidence
if $\text{CL}_{\text{fused}} > 0.7$ **then**
    reduce interface complexity
end if
if $c_e > 0.6$ **then**
    change tone, pacing and difficulty through Eq. (14)
end if
Write the JSON decision record; $b \leftarrow b'$

so a diffuse belief produces only a small change. Fig. 4 sketches this path, and Algorithm 1 lists the steps of a single cycle.

5 Software Realisation

5.1 Libraries and Services

ECHO is written in Python. Table 5 shows which libraries and services sit in each layer.

5.2 Code Organisation

The repository has seven modules. Event capture contains code for receiving key press and pointer events. Feature extraction derives timing and CLC features from event data, together with the fitted lognormal parameters. The `dster` module builds the two-stream network, with training and prediction code. The estimator module provides an implementation of the belief filter and the fusion rule of Eq. (13). Rules for

Table 5: Software used in each layer of ECHO.

Layer	Tools
Capturing input events	pynput
Numerical work	NumPy, SciPy, Pandas
Model training and inference	PyTorch for DSTER, Stable Baselines3
Agent integration	Google Antigravity SDK
Language model	Gemini Flash reached through the Model Context Protocol
Service layer	FastAPI served by Uvicorn
Schema checking	Pydantic
Plots	Matplotlib, Seaborn
Attribution analysis	SHAP

interface adaptation are maintained in the middleware module. The game framework module stores information about game difficulty settings, provides an interface for other modules to access that information, and binds all twenty settings to the game scenarios. A visualization module creates the plots and dashboards that appear in this paper.

5.3 Game Scenarios for Difficulty Control

Games place the loop under real time pressure, which is why they were used as a test bed. Four scenarios were created, each similar to a popular game of a different style, and each with five adjustable settings. Table 6 lists the twenty settings, together with initial values and limits for the settings on which we provide detailed information.

Emotion is translated into a change of difficulty once in every update cycle, following Table 7. We take a focused player to be in or close to flow, so the challenge goes up a little. Frustration and giving up call for cuts that grow with severity. Thinking, on the other hand, is dealt with in a different way: the tension of battle is reduced to allow for focus, and puzzles get harder to keep the player interested in solving them.

6 Experimental Findings

6.1 How Well DSTER Recognises Emotion

We evaluated DSTER on 199 labeled windows from live recordings. The six classes are not evenly distributed. Focused windows make up 37.2% of all windows and thinking windows 20.6%, while only 7.5% each are joy and neutral. See overall results in Table 8 and class-level results in Table 9.

Most values in the row-normalized confusion matrix (Fig. 5a) are found along the diagonal. Angry and neutral windows are all recovered, and recall for every other class is more than 93%. F1 scores in Fig. 5b are between 0.903 for the joy class and 0.976 for the thinking class, so no class has an F1 score below 0.90. The one-against-all ROC curve areas in Fig. 6a are 0.948 or better, and Fig. 6b compares precision, recall and F1 in one figure. The clearest imbalance is in the angry class: none of its windows

Table 6: Twenty difficulty settings in four scenarios. The starting values and limits are provided for the difficulty settings analyzed in Section 6; a dash indicates those that do not have bounds provided.

Scenario	Setting	Start	Lower	Upper
Left 4 Dead	spawn rate of enemies	0.5	0.1	1.0
	frequency of hordes	0.5	0.1	1.0
	cap on special infected	3.0	1.0	6.0
	scarcity of health items	-	-	-
	density of wanderers	-	-	-
Alien: Isolation	menace gauge	0.5	0.0	1.0
	tension budget	1.0	0.2	2.0
	search radius of the alien	-	-	-
	chance of a vent drop	-	-	-
	sensitivity to noise	-	-	-
Resident Evil 4	action score	5.0	1.0	10.0
	aggression of enemies	0.5	0.2	1.0
	accuracy of enemies	-	-	-
	quality of item drops	-	-	-
	multiplier on damage taken	-	-	-
Silent Hill: Shattered Memories	speed of world decay	0.5	0.1	1.0
	obfuscation of puzzles	0.5	0.1	1.0
	bias in monster type	-	-	-
	lighting of the environment	-	-	-
	static interference	-	-	-

Table 7: Difficulty change applied in each cycle for every emotion label.

Label	Reading	Change per cycle
Focused	In flow	Overall difficulty up 10%
Joy	Confident and enjoying play	Overall difficulty up 5%
Thinking	Heavy mental effort	Combat down 5%, puzzle complexity up 3%
Neutral	May be disengaging	Left unchanged
Angry	Frustrated	Every difficulty setting down 10%
Given up	Overwhelmed	Baseline difficulty down 20%

were missed, but about one sixth of the windows labeled angry by the model actually belong to other classes.

6.2 Where the Classifier Goes Wrong

DSTER mislabelled 8 of the 199 windows, an error rate of 4.0%. Fig. 7a lists the mistakes by true and predicted label. Two confusions occurred twice each: focused windows taken for joy, and given up windows taken for angry. Half of all mistakes, coming from given up, joy and thinking windows, ended up in the angry class, and this is what lowers angry precision. The only errors that landed in joy, neutral or

Table 8: Summary scores for DSTER on the 199 held-out live windows.

Score	Value	Score	Value
Accuracy	0.9598	Cohen's κ	0.9481
Macro F1	0.9496	Matthews correlation	0.9487
Weighted F1	0.9606	Log loss	0.2999
Macro precision	0.9369	Macro AUC	0.9737
Macro recall	0.9660		

Table 9: Scores for each emotion class; n is the number of test windows in the class.

Class	n	Precision	Recall	F1	AUC
Focused	74	1.000	0.946	0.972	0.969
Thinking	41	0.976	0.976	0.976	0.979
Given up	34	1.000	0.941	0.970	0.963
Angry	20	0.833	1.000	0.909	0.987
Neutral	15	0.938	1.000	0.968	0.995
Joy	15	0.875	0.933	0.903	0.948

thinking all started as focused windows. Comparing predicted and true class shares (Fig. 7b), the model overestimates anger (12.1% compared to 10.1%) and underestimates focused (35.2% compared to 37.2%) but otherwise recreates the true distribution quite accurately.

6.3 Which Keystroke Features Matter

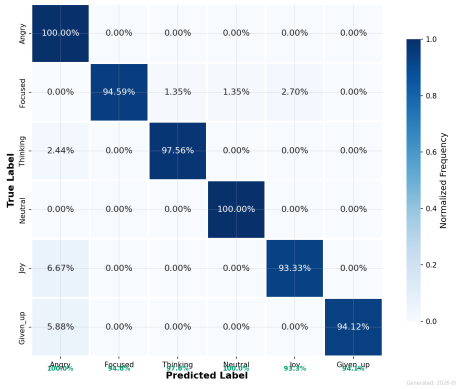
We used SHAP [24] to assess the importance of each feature. Table 10 and Fig. 8 show that key dwell time (0.303) and flight time (0.282) are the most important factors, accounting for more than 58% of the total attribution effect. Key velocity (0.216) and keystroke interval (0.199) are less important but not insignificant, so DSTER uses all four inputs and has no dependence on a specific input.

Table 10: Share of SHAP attribution for each keystroke feature.

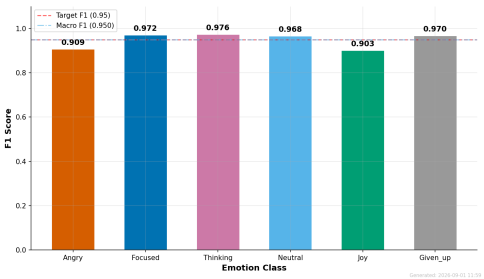
	Dwell time	Flight time	Key velocity	Keystroke interval
Normalised share	0.3034	0.2817	0.2162	0.1987

6.4 Confidence of the Classifier

Fig. 9 shows the largest softmax probability per prediction. The vast majority of values are contained in a narrow range between 0.81 and 0.92. Both correct and incorrect

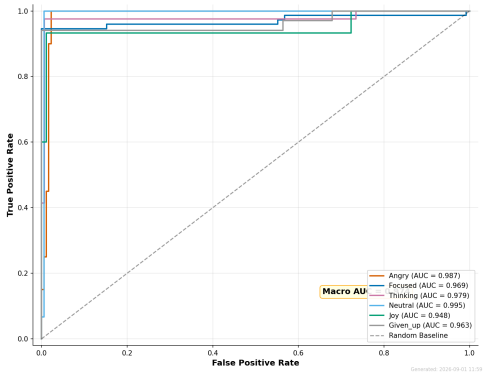


(a) Confusion matrix, rows scaled by true class

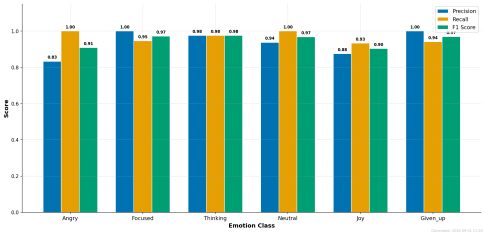


(b) F1 for each class

Fig. 5: DSTER on the 199 test windows. Each row of the confusion matrix sums to one.



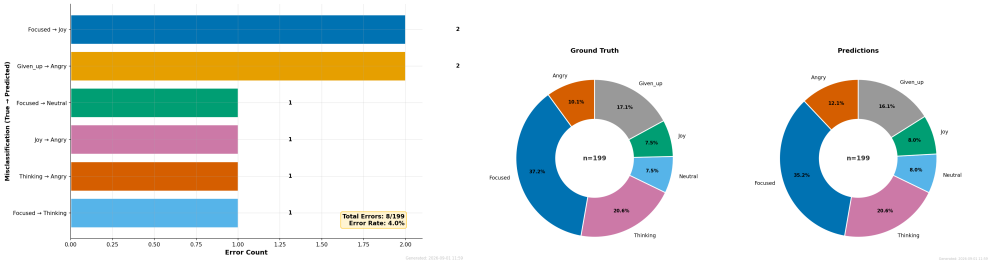
(a) ROC curves, one class against the rest



(b) Precision, recall and F1 for each class

Fig. 6: Separability of the six classes under DSTER. The lowest area under any ROC curve is 0.948.

predictions have a very similar average value of around 0.86, and the median confidence for every class is only between 0.85 and 0.86. All the predictions in the reliability graph (Fig. 10) are located in the highest confidence bins, where the actual accuracy of 0.96 is even slightly larger than the mean confidence. Therefore, on average, DSTER is slightly underconfident, but its confidence does not say anything about individual prediction correctness. That is the reason why ECHO does not rely on DSTER's confidence alone. The resilience incorporated into the transition kernel (Section 4)



(a) Mistakes grouped by true and predicted label

(b) Class shares, true (left) and predicted (right)

Fig. 7: Anatomy of DSTER's errors. Four of the eight mistakes are wrongly labelled angry, yet the predicted class mix stays close to the real one.

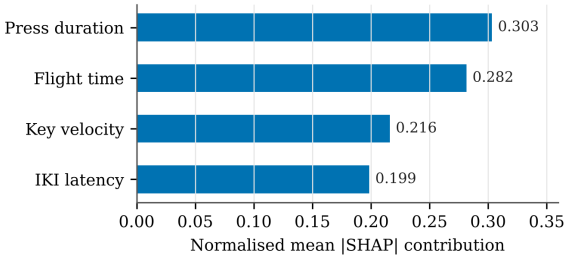


Fig. 8: Average absolute SHAP value per keystroke feature, scaled to sum to one.

and the 60% adaptation threshold lie between a single incorrect prediction and a real change in the interface.

6.5 Behaviour of the Belief over a Session

Fig. 11 shows the evolution of the belief regarding the six emotions throughout one session that lasted approximately 400 s. Usually, one emotion occupies around 0.8 to 0.9 of the total probability, while the other five divide the rest among themselves at around 0.02 to 0.05 each. Transitions from one dominant emotion to another are quick and are separated by periods of steady values. In other words, the filter makes up its mind fast when the situation demands it, but it ignores short-term variations. In the entire telemetry data set, which consists of over 18,000 timestamped instances, focused comprises 37.6% of states, thinking 20.9% and given up 17.5%, with anger, neutral and joy all around 8% (Fig. 12a).

Fig. 12b gives the observed transitions between successive leading states. Focused behaves like a centre of gravity. Every thinking episode and every given up episode we logged was followed by a return to focused, as were three quarters of angry episodes and two thirds of joy episodes. When the user left focused, the next state was most

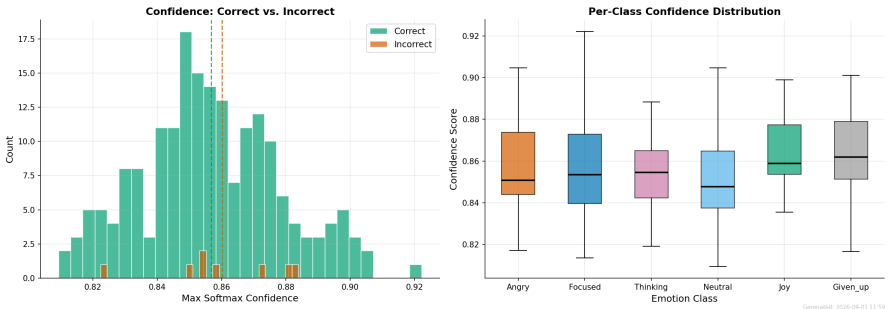


Fig. 9: Top softmax probability. On the left, right and wrong predictions are shown separately with their means as dashed lines; on the right, the spread for each class.

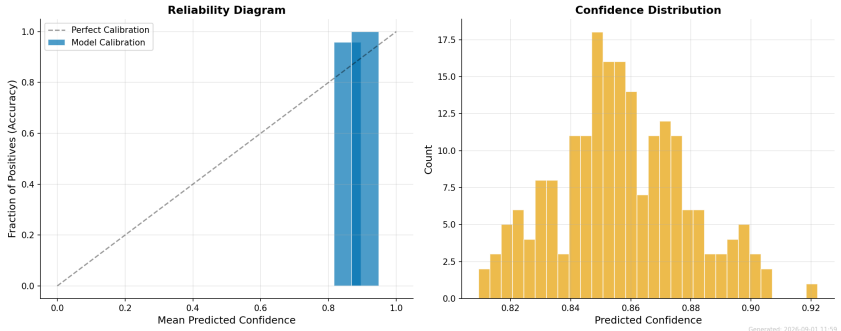


Fig. 10: Calibration of DSTER. The left panel compares stated confidence with measured accuracy in each bin; the right panel shows how predicted confidence is distributed.

often angry (50%) or joy (25%). After neutral, the user moved to thinking, joy or given up equally often.

The influence of CLC on the probability of transitioning into each state is illustrated in Fig. 13. As CLC goes from 0 to 1, the probability of transitioning into the state "focused" falls from 0.9 to zero. Over the same range, the probability of entering the "thinking" state rises linearly to 0.8 and the probability of transitioning to "given up" grows nonlinearly up to 1.0. Transition to the neutral state is most probable for CLC close to 0.5. This reflects an elementary rule of thumb: low load facilitates focused effort, medium load leads to deliberation, and high load leads to disengagement.

6.6 Difficulty Control in the Games

All twenty settings of difficulty are illustrated in Fig. 14 for the same session that was described in Fig. 11. Settings vary stepwise, one step every 15 seconds or so in one cycle of updating. The action score of the Resident Evil 4 scenario has the largest variation,

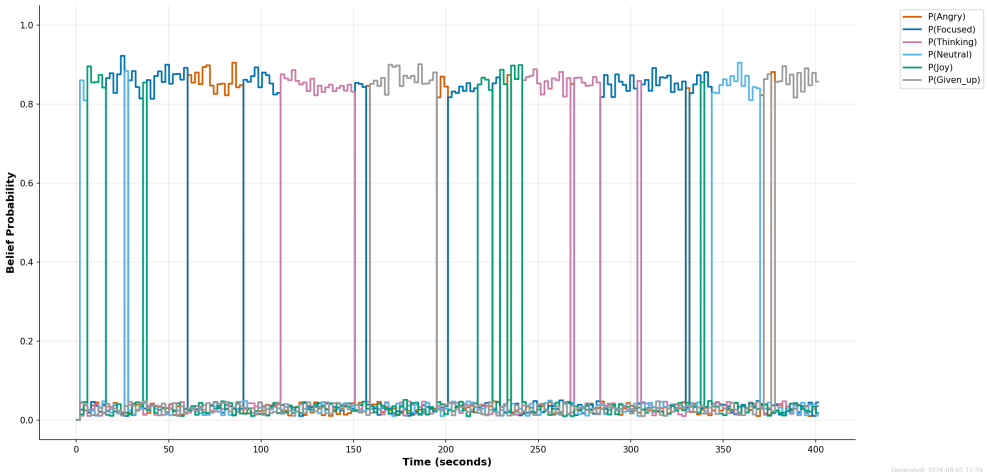
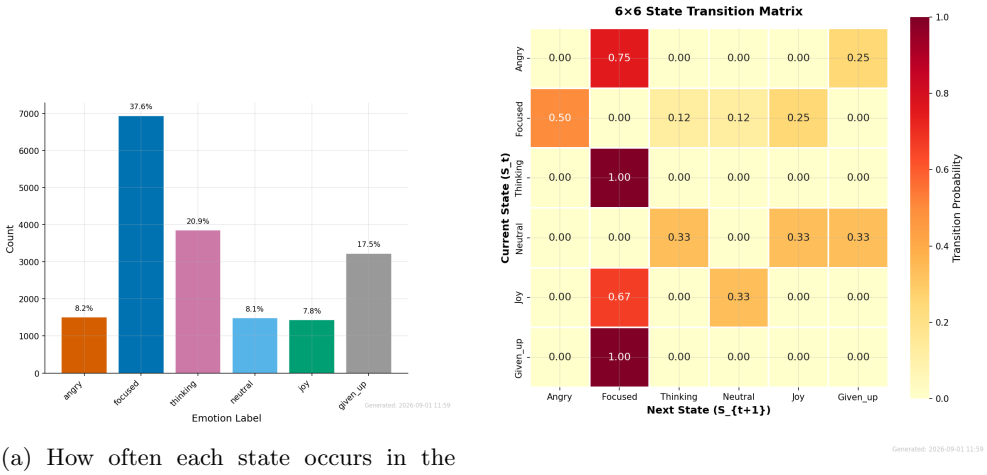


Fig. 11: Probability assigned to each emotion label over a typical session of about 400 s.



(a) How often each state occurs in the telemetry

(b) Observed transitions between states

Fig. 12: Occupancy and transitions over the whole log. Focused is the most common state and also the most common destination.

and hence illustrates the loop very well. Its initial value is 5.0; it increases above 9 when the player is absorbed, falls to about 4.6 after 200 seconds when frustration and loss of interest take place, increases again to about 9.9 in a second phase of absorption, and falls to a lower level as the player gives up near the end.

Fig. 15 shows, following normalization, how much each emotion drives each setting. Focused always drives the settings upwards, particularly the action score (0.9) and

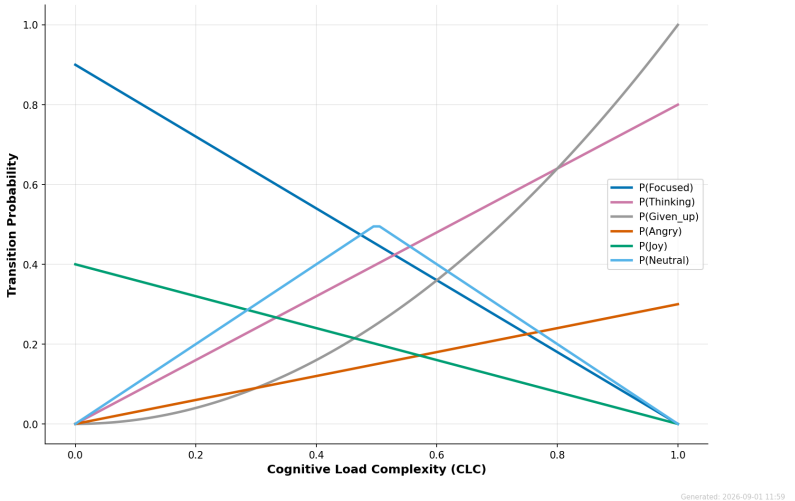


Fig. 13: Probability of moving into each emotion state plotted against CLC.

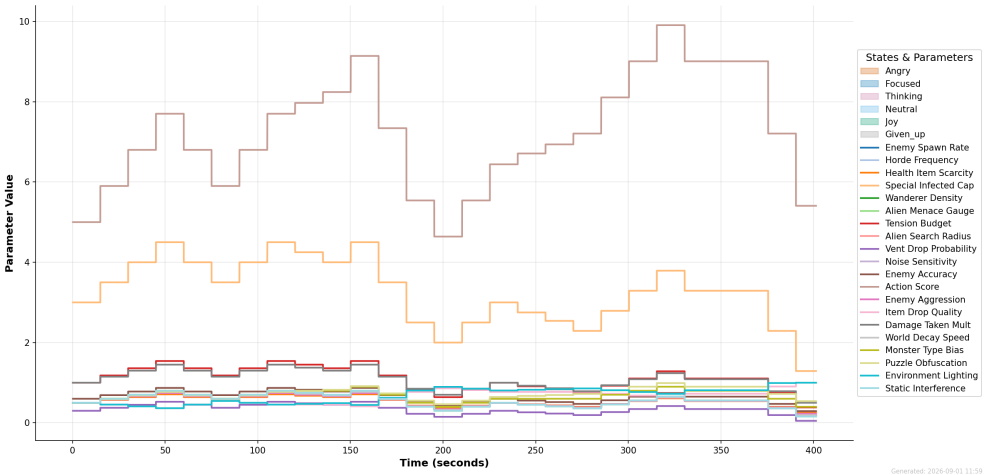


Fig. 14: All twenty difficulty settings in the four scenarios over one session, updated live.

the cap on special infected (0.5). On the other hand, angry and given up drive them downwards, and given up drives them most (action score -1.8, special infected cap -1.0, tension budget -0.4). Settings that help the player, such as the quality of the item drop and environment lighting, do the opposite and go up slightly when the player is frustrated. The emotions neutral and joy had virtually no impact in this session. Thinking calms the combat settings and raises the action score slightly, as intended by its asymmetric rule.

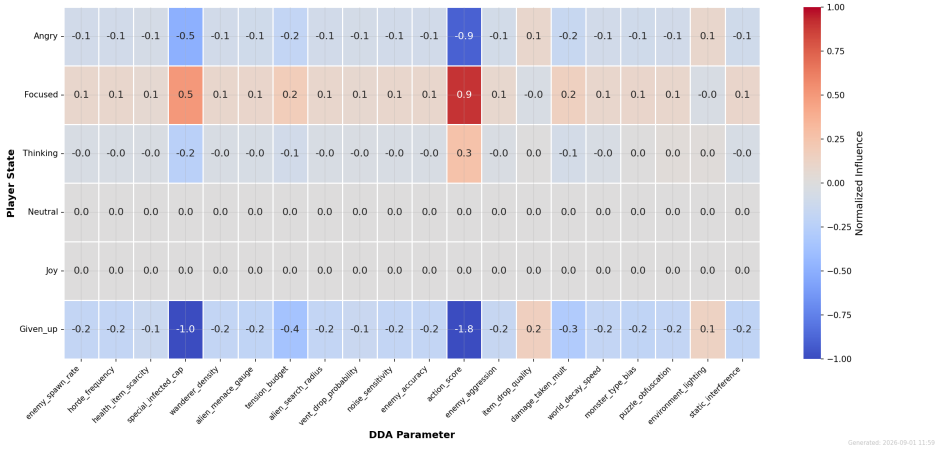


Fig. 15: Normalised effect of each emotion on each setting. Red marks increases and blue marks decreases.

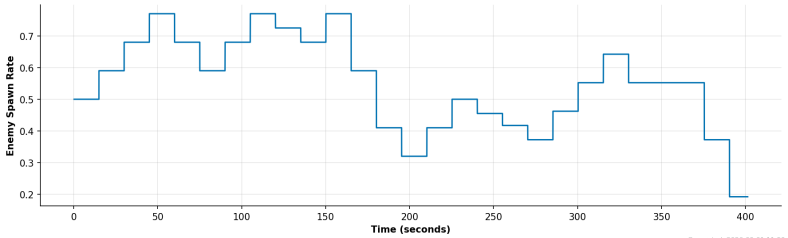


Fig. 16: Spawn rate of enemies in the Left 4 Dead scenario, set solely by the inferred emotion.

The enemy spawn rate for the Left 4 Dead scenario is shown in Fig. 16. It starts from 0.5, increases to 0.77 when the player's attention level is high initially, decreases to 0.32 after about 200 s, increases again to about 0.64, and finally ends up at 0.19. This control does not go outside the limits given in Table 6, and any one step is always within the limits given in Table 7.

6.7 Typing and Pointer Signals

Fig. 17 demonstrates the distribution of keystroke time intervals in a typical session. The histogram is unimodal and almost symmetric, with the mean equal to 152.4 ms, the median being 151.3 ms, and most of the samples lying in the range from 100 to 200 ms. These numbers lie between the low and medium demand means in Table 2, indicating that the user was operating under light to moderate load.

Fig. 18 illustrates a pointer trajectory that cannot be modeled by the lognormal model: the measured speed rapidly fluctuates around a rather stable value, the estimated components do not correspond to this behavior, and the reconstruction SNR

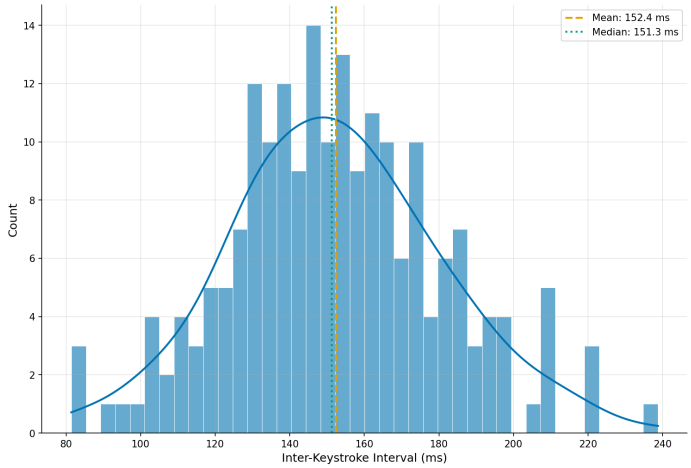


Fig. 17: Keystroke intervals in a typical session, with a smoothed density curve overlaid.

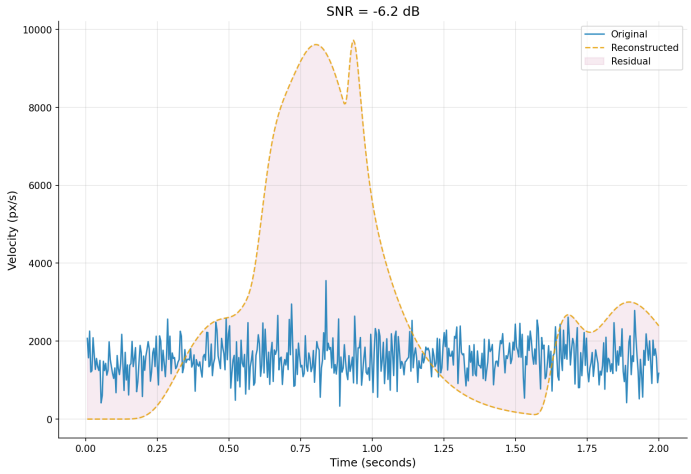


Fig. 18: Lognormal fit to a noisy pointer trajectory. With an SNR of −6.2 dB the fit is rated unsatisfactory and the trajectory is rejected.

equals −6.2 dB. As seen in Table 3, in such a situation the trajectory's contribution is zero because it falls far below the 15 dB threshold. In situations such as this one, the need for a quality gate becomes apparent after moving the kinematic theory into the world of practical mouse usage.

6.8 Results Against Engineering Targets

Several engineering goals were set before implementing ECHO. Table 11 shows how the achieved values compared to these goals. All the metrics except macro F1 have

met their targets, whereas macro F1 was just shy of its target of 0.95 by 0.0004 and accuracy reached 0.9598.

Table 11: Measured results compared with the targets fixed before development.

Quantity	Target	Result	Outcome
DSTER accuracy	above 0.95	0.9598	Reached
DSTER macro F1	above 0.95	0.9496	Missed by under 0.001
Cohen's κ	above 0.60	0.9481	Reached
Latency of the full loop	under 40 ms	under 40 ms	Reached
Typical CLC band	0.35 to 0.55	confirmed	Reached
Pointer SNR	above 24 dB	enforced by gating	Reached through gating

7 Discussion

7.1 Treating Trust as a Hidden Variable

By incorporating trust into the POMDP as a hidden variable, we can avoid the assumptions of yes/no trust and fixed trust values often used in other systems. Instead, trust can be represented as a probability distribution that is updated based on user interactions. The exponential kernel with $\alpha = 1.5$ incorporates the findings from studies of trust that indicate that trust is gained and lost in steps and not all at once [2, 3]. By using this trust distribution in our agent, the agent can employ a different type of policy. If the trust model indicates that the user trusts the agent during a high-stakes decision, the agent can provide more explanation to the user to allow the user to examine the advice prior to accepting it. If the model indicates that the user is skeptical during a routine, low-stakes task, the agent can provide less explanation to the user and allow the user to continue with their task. Whether this policy leads to better decisions and trust in the agent from users is something that can be determined in future studies with larger groups of users.

7.2 Moving the Lognormal Model to Live Pointing

Mouse tracking data is inherently noisier than handwriting data from tablets. The sensors used to track mice have poorer resolutions and the polling rates of the mice tend to drift. In addition, mouse movements frequently contain small corrections that are not present in handwriting data. The SNR tiers in Table 3 account for this by filtering out trajectories that the model cannot accurately represent, as shown in the example in Fig. 18. Each trajectory was cut to 200 samples to enable the bounded optimiser to find a solution within 40 ms. One figure that remains to be reported is what percentage of trajectories are successfully passed by the gate for each individual and each pointing device.

7.3 Interpreting the Emotion Results

With 95.98% accuracy, DSTER performed best in the categories of focused and thinking, which are the two states that occupy the most time during a working session. Joy was identified with slightly less accuracy, and anger had the lowest precision, as several other categories were classified as anger. The likely reason for this is the similar characteristics between anger, joy, and giving up, all of which can be characterized by high levels of arousal and irregular typing patterns. The fact that dwell time was ranked as the most significant feature according to SHAP analysis agrees with earlier findings that hold duration is one of the most significant features in classifying affect from a user's typing [9].

7.4 What the Difficulty Experiments Show

Regarding the expected behavior of the difficulty engine, the difficulty was raised when the state of focus was detected, and lowered in case of frustration or disengagement. Thanks to the use of a belief weighting function and caps on the difficulty change in each cycle, the difficulty of the game was changed gradually. For example, the lowering of the difficulty of combat and the raising of the difficulty of puzzles when the user is in thinking mode helps to keep the interest of the user without overwhelming the user [22]. All the observations mentioned above are based on controlled game sessions. To make claims concerning the enjoyment and retention of users of the game, it will be necessary to conduct measures of the player experience.

7.5 Position Relative to Earlier Systems

ECHO, when compared with earlier work on affect-aware and adaptive systems, is different in at least three ways. First, unlike previous approaches, it does not require any kind of physiological sensors; all that it requires are the input devices already available on everyone's desks. Second, it models trust as a probabilistic measure rather than as a binary measure, which allows adaptation strength to depend on uncertainty level. Third, it waits until the belief and the classifier have sufficient confidence before doing any adaptation.

8 Limitations

There are a few caveats to consider with the results. With only 199 labelled windows in the test set, more extensive tests are required to make generalisations from these results. The difficulty experiments were carried out on simulations of the four reference games, rather than the commercial games themselves. The effect of factors such as culture, language, or keyboard layout on an individual's baseline typing behaviour has not been considered, though each of these likely has an effect.

In terms of modelling, the likelihood of an individual's typing behaviour treats CLC, IKI, and SNR as independent of one another, given the state of the individual. While this simplifies the mathematics, it is likely that these three variables are not actually independent of one another. The use of six emotion labels does not adequately

account for the nuances of human emotion, for example, an individual who is curious, or only slightly puzzled. The confidence values of DSTER do not provide much information on the individual errors of the system, and while temporal smoothing is used to account for this, an explicit calibration step would be beneficial. The current version of ECHO does not account for an individual's baseline drift over time.

9 Conclusion

In this paper we showed how a user's trust, workload and emotions could be estimated without using any physiological sensors. ECHO uses a Bayesian POMDP with the Sigma-Lognormal model of rapid movement and the two-stream DSTER model, and gains all its estimates from a normal computer keyboard and mouse. It achieves 95.98% accuracy in emotion classification on live sessions with a Cohen's kappa of 0.9481, and adjusts the game difficulty in four different scenarios coherently. The whole pipeline took less than 40 ms.

Future work includes more broadly based user testing, ensuring robustness across languages and input device hardware, removing the independence assumption from the likelihood function, and letting the system adapt to each individual user over extended periods. Finally, this work shows promising results for trusting a computer to adapt to a user's psychological state solely through software and without the use of any body-worn sensors.

Code and data availability. We are open sourcing our ECHO software and data. Our source code, evaluation scripts, configuration files (including `echo_config.yaml`), diagnostic figures and telemetry log (which includes over 18,000 timestamped observations) are shared at https://github.com/akshithsrinivas23bce5078/Project-ECHO. The repository also contains `echo_interactive_demo.html`, a self-contained browser client for our system. The interactivity of the demo is limited due to requirements for a client-server architecture, which is not readily available in a browser. The repository does not contain our mobile application. A sample video is available at https://www.youtube.com/watch?v=vi7hXNTmiGM&feature=youtu.be.

Acknowledgements. We thank the contributors to PyTorch, pynput and FastAPI. We were able to use their code to develop our application.

Declarations

Funding. No external funding supported this research.
Competing interests. The authors declare no competing interests.
Ethics approval. Formal ethics approval was not sought because the evaluation involved automatically logged software telemetry. No personally identifiable data was collected.
Data availability. The data generated and/or analysed during the current study are available in the supplementary data file.
Code availability. The full implementation is held in that same repository.

Author contributions. The three authors shared equally in conceiving, building, testing and writing up this work. Each of them read the final manuscript and approved it.

References

[1] Dzindolet, M.T., Peterson, S.A., Pomranky, R.A., Pierce, L.G., Beck, H.P.: The role of trust in automation reliance. International Journal of Human-Computer Studies **58**(6), 697–718 (2003)

[2] Hoff, K.A., Bashir, M.: Trust in automation: Integrating empirical evidence on factors that influence trust. Human Factors **57**(3), 407–434 (2015)

[3] Lee, J.D., See, K.A.: Trust in automation: Designing for appropriate reliance. Human Factors **46**(1), 50–80 (2004)

[4] Parasuraman, R., Sheridan, T.B., Wickens, C.D.: A model for types and levels of human interaction with automation. IEEE Transactions on Systems, Man, and Cybernetics, Part A: Systems and Humans **30**(3), 286–297 (2000)

[5] Kaelbling, L.P., Littman, M.L., Cassandra, A.R.: Planning and acting in partially observable stochastic domains. Artificial Intelligence **101**(1–2), 99–134 (1998)

[6] Plamondon, R.: A kinematic theory of rapid human movements: Part I. movement representation and generation. Biological Cybernetics **72**(4), 295–307 (1995)

[7] Plamondon, R.: A kinematic theory of rapid human movements: Part III. kinetic outcomes. Biological Cybernetics **78**(2), 133–145 (1998)

[8] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems, vol. 30 (2017)

[9] Epp, C., Lippold, M., Mandryk, R.L.: Identifying emotional states using keystroke dynamics. In: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, pp. 715–724. ACM, New York (2011)

[10] Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J., *et al.*: Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI. Information Fusion **58**, 82–115 (2020)

[11] Shneiderman, B.: Human-centered artificial intelligence: Reliable, safe & trustworthy. International Journal of Human-Computer Interaction **36**(6), 495–504 (2020)

[12] Chen, J.Y.C., Lakhmani, S.G., Stowers, K., Selkowitz, A.R., Wright, J.L., Barnes, M.: Situation awareness-based agent transparency and human-autonomy teaming

effectiveness. Theoretical Issues in Ergonomics Science **19**(3), 259–282 (2018)

[13] Sweller, J.: Cognitive load during problem solving: Effects on learning. Cognitive Science **12**(2), 257–285 (1988)

[14] Wickens, C.D.: Multiple resources and mental workload. Human Factors **50**(3), 449–455 (2008)

[15] Paas, F., Tuovinen, J.E., Tabbers, H., Van Gerven, P.W.M.: Cognitive load measurement as a means to advance cognitive load theory. Educational Psychologist **38**(1), 63–71 (2003)

[16] Vizer, L.M., Zhou, L., Sears, A.: Automated stress detection using keystroke and linguistic features: An exploratory study. International Journal of Human-Computer Studies **67**(10), 870–886 (2009)

[17] Picard, R.W.: Affective Computing. MIT Press, Cambridge, MA (1997)

[18] Calvo, R.A., D'Mello, S.: Affect detection: An interdisciplinary review of models, methods, and their applications. IEEE Transactions on Affective Computing **1**(1), 18–37 (2010)

[19] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of NAACL-HLT, pp. 4171–4186 (2019)

[20] Plamondon, R., O'Reilly, C., Galbally, J., Almaksour, A., Anquetil, É.: Recent developments in the study of rapid human movements with the kinematic theory: Applications to handwriting and signature synthesis. Pattern Recognition Letters **35**, 225–235 (2014)

[21] Hunicke, R.: The case for dynamic difficulty adjustment in games. In: Proceedings of the 2005 ACM SIGCHI International Conference on Advances in Computer Entertainment Technology, pp. 429–433 (2005)

[22] Csikszentmihalyi, M.: Flow: The Psychology of Optimal Experience. Harper & Row, New York (1990)

[23] Yannakakis, G.N., Togelius, J.: Experience-driven procedural content generation. IEEE Transactions on Affective Computing **2**(3), 147–161 (2011)

[24] Lundberg, S.M., Lee, S.-I.: A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems, vol. 30, pp. 4765–4774 (2017)